NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & IT

TS (COMPUTER SCIENCE)

CT-263 OBJECT-ORIENTED ANALYSIS AND DESIGN

REPORT

SmartRide: Ride Sharing and Carpooling Platform

Muhammad Obaid	CT-23025
Akhyar Ahmed Turk	CT-23034
Muhammad Rehan	CT-23039
Muhammad Affan	CT-23040

Table of Contents

1. Project Overview 5
   1.1 Purpose, Scope & Objectives 5
   1.2. Assumptions and Constraints 6
   1.3. Project Deliverables 7
2. System Analysis 8
   2.1. Requirements Specification 8
      2.1.1. Functional Requirements: 8
      2.1.2. Non-Functional Requirements: 11
      2.1.3. Operational Scenarios: 15
   2.2. Structural Model 16
      2.2.1. Class Diagram 16
      2.2.2. Object Diagram 17
      2.2.3. Component Diagram 18
      2.2.4. Deployment Diagram 19
   2.3. Behavioral Model 20
      2.3.1. Use Case Diagram 20
      2.3.2. Activity Diagram 21
      2.3.3. Sequence Diagram 24
      2.3.4. Communication Diagram 25
      2.3.5. State Diagram 26
3. System Design 29
   3.1. System Architecture 29
      1. Client Side: 30
      2. Server Side: 31
      3. Business Logic: 33
      4. External Services: 34
   3.2. Design Patterns 35
   Observer Pattern 35
4. Test Cases 39
   4.1. Unit Testing 39
   4.2. Blackbox Testing 41
   4.3. Integration Testing 43
   4.4. Regression Testing 45
   4.5. Validation Testing 46

4.6. Performance Testing 48
4.7. Load & Stress Testing 49
4.8. User Acceptance Testing 51
4.9. Alpha & Beta Testing 53
Appendix A 55
Definitions 55
Acronyms 56
Abbreviations 56
Appendix B 57
References 57

1.Project Overview

1.1 *Purpose, Scope & Objectives*

Purpose:

This document provides a complete description of all functions and specifications of the SmartRide ride-sharing and carpooling platform. The document is intended for:

- ➔ Development team members responsible for implementing the system
- ➔ Project managers overseeing the development process
- ➔ Quality assurance team members conducting testing
- ➔ Stakeholders evaluating project progress and deliverables

This document establishes the foundation for all subsequent design, development, and testing activities.

Scope:

SmartRide is a comprehensive ride-sharing and carpooling platform designed to connect drivers offering rides with passengers seeking transportation. The system encompasses the following key capabilities:

System Boundaries:

- ➔ User registration and authentication for drivers and passengers
- ➔ Real-time ride request and offer management
- ➔ Intelligent matching algorithm based on proximity, destination, and capacity
- ➔ Route optimization and navigation integration
- ➔ In-app communication between drivers and passengers
- ➔ Payment processing and transaction management
- ➔ Rating and feedback system for service quality
- ➔ Trip history and analytics

Benefits:

- ➔ Reduced traffic congestion through shared rides
- ➔ Lower transportation costs for users
- ➔ Environmental benefits through reduced carbon emissions
- ➔ Efficient utilization of vehicle capacity
- ➔ Enhanced urban mobility solutions

Out of Scope:

- ➔ Vehicle maintenance tracking
- ➔ Insurance claim processing
- ➔ Emergency medical services integration
- ➔ Multi-city cross-border rides *(Phase 1)*

Objectives:

The objective of the SmartRide platform is to provide a reliable, safe, and efficient ride-sharing and carpooling solution that connects drivers and passengers in real time. The system aims to reduce travel costs, minimize traffic congestion, optimize vehicle usage, and offer a seamless transportation experience through intelligent ride matching, secure payments, and user-friendly interfaces.

1.2. *Assumptions and Constraints*

Assumptions:

- ➔ Users have access to smartphones or computers with internet connectivity
- ➔ Users grant location access permissions to the application
- ➔ Drivers have valid licenses and vehicle insurance
- ➔ Payment gateway services remain available and operational
- ➔ Map and navigation services are accessible
- ➔ Users have basic literacy and smartphone operation skills

Constraints:

Technical Constraints:

- ➔ Must support Android 8.0+ and iOS 12+ for mobile applications
- ➔ Web application must support Chrome, Firefox, Safari, and Edge browsers
- ➔ Maximum response time of 3 seconds for ride matching
- ➔ GPS accuracy within 10 meters for location tracking
- ➔ Must handle concurrent users in the range of 5000-10,000.

Business Constraints:

- ➔ Development budget limited to available resources
- ➔ Must comply with local transportation regulations
- ➔ Project timeline: 5-8 weeks for Phase 1 development
- ➔ Limited to single-city operation initially

Regulatory Constraints:

- ➔ Data privacy compliance (GDPR-equivalent standards)
- ➔ Payment Card Industry (PCI) compliance for transactions
- ➔ Local transportation authority licensing requirements
- ➔ Age verification requirements for drivers and passengers

1.3. *Project Deliverables*

The SmartRide project will deliver the following core system components and functionalities:

- ➔ A fully functional **ride-sharing and carpooling application** that connects drivers and passengers in real time.
- ➔ **User Registration & Authentication Module** for secure login, role selection, and driver document verification.
- ➔ **Ride Request & Offer System** enabling passengers to request trips and drivers to post available rides.

- **Real-time Matching Algorithm** that automatically pairs passengers with suitable drivers based on proximity, destination, timing, and seat availability.
- **GPS Integration** for live route tracking, map display, and optimized routing.
- **In-App Notification Features** including messaging, notifications, and emergency contact options.
- **Fare Calculation & Payment Module** supporting secure transactions, wallet management, and trip receipts.
- **Rating & Feedback System** for maintaining service quality and improving user trust.
- **Admin Dashboard** for monitoring users, managing disputes, and overseeing system performance.
- **Trip History & Analytics Module** providing insights into past rides, spending, earnings, and usage patterns.
- **Secure Cloud-Based Database** storing user data, ride information, payment records, and logs.

2. System Analysis

2.1. *Requirements Specification*

2.1.1. Functional Requirements:

The SmartRide system shall provide the following functional requirements categorized by major system modules:

User Account Management:

- **FR-1:** The system shall allow users to create an account as either a Passenger or Driver.
- **FR-2:** The system shall authenticate users using email/phone and password.

- **FR-3:** The system shall allow drivers to upload license and vehicle documents for verification.
- **FR-4:** The system shall allow users to update their profile information at any time.
- **FR-5:** The system shall allow administrators to approve or reject driver verification requests.

Ride Request (Passenger Side):

- **FR-6:** The system shall allow passengers to request a ride by entering pickup and destination locations.
- **FR-7:** The system shall display estimated fare to the passenger before ride confirmation.
- **FR-8:** The system shall send the ride request to multiple nearby drivers automatically.
- **FR-9:** The system shall notify the passenger when a driver accepts or rejects a request.

Ride Create (Driver Side):

- **FR-10:** The system shall allow drivers to offer a ride by setting starting point, destination, and available seats.
- **FR-11:** The system shall show incoming ride requests to the driver.
- **FR-12:** The system shall allow the driver to accept or decline ride requests.
- **FR-13:** The system shall update seat availability after each accepted passenger.

Ride Matching & Allocation:

- **FR-14:** The system shall match passengers with drivers based on proximity, route similarity, and available seats.
- **FR-15:** The system shall notify both users once a match is confirmed.

Real-Time Tracking & Navigation:

- ➔ **FR-17:** The system shall display real-time driver location on the passenger's map.
- ➔ **FR-18:** The system shall provide estimated time of arrival (ETA) to both users.
- ➔ **FR-19:** The system shall update the trip route continuously using GPS.

Communication:

- ➔ **FR-20:** The system shall allow drivers and passengers to chat via in-app messaging.
- ➔ **FR-21:** The system shall send push notifications for ride acceptance, cancellation, arrival, payment, and rating.
- ➔ **FR-22:** The system shall allow passengers to call the driver via masked (hidden) contact numbers.

Payment & Fare Management:

- ➔ **FR-23:** The system shall calculate ride fare based on distance, route, and ride-sharing split.
- ➔ **FR-24:** The system shall allow users to pay via integrated payment gateway or in-app wallet.
- ➔ **FR-25:** The system shall generate digital receipts after each trip.
- ➔ **FR-26:** The system shall maintain a wallet history for both passengers and drivers.

Rating & Feedback:

- ➔ **FR-27:** The system shall allow passengers and drivers to rate each other after the trip.
- ➔ **FR-28:** The system shall allow users to submit written feedback or complaints.
- ➔ **FR-29:** The system shall calculate average ratings for profiles.

Admin Features:

- **FR-30:** The system shall allow the admin to view all users, trips, payments, and reports.
- **FR-31:** The admin shall be able to suspend/ban a user violating rules.
- **FR-32:** The admin shall be able to resolve disputes and complaints.

Trip History & Analytics:

- **FR-33:** The system shall store all completed trip details for future reference.
- **FR-34:** The system shall allow users to view previous rides, payments, and ratings.
- **FR-35:** The system shall generate analytical reports for admin (usage, peak time, number of trips, etc.).

2.1.2. Non-Functional Requirements:

a) Performance Requirements:

Response Time

Requirement:

The system shall respond to user requests within **3 seconds** under normal conditions.

Priority:

High

Justification:

Ensures smooth and responsive user experience.

Concurrent Users

Requirement:

The system shall support up to **10,000 concurrent users**.

Priority:

High

Justification**:**

Allows scalability for a large user base.

b) Security Requirements:

Secure Communication

Requirement**:**

All communication between client and server shall be encrypted using **HTTPS with TLS 1.2+**.

Priority:

High

Justification:

Protects user data from interception or misuse.

Password Security

Requirement:

User passwords shall be stored using secure encryption and hashing mechanisms.

Priority:

High

Justification:

Safeguards sensitive user credentials.

c) Usability Requirements:

User Interface Quality

Requirement:

The system interface shall be simple, intuitive, and mobile-responsive.

Priority:

High

Justification:

Improves accessibility for diverse users.

d) Reliability Requirements:

System Availability

Requirement:

The platform shall maintain **99% uptime** during operational hours.

Priority**:**

Medium

Justification:

Guarantees continuous service availability.

Data Backup

Requirement:

The system shall perform daily data backups to prevent data loss.

Priority:

Medium

Justification:

Ensures recovery after unexpected failures.

e) Maintainability Requirements:

Code Maintainability

Requirement:

The codebase shall be modular and well-documented.

Priority:

Medium

Justification:

Simplifies updates and debugging.

f) Portability Requirements:

Browser Compatibility

Requirement:

The web app shall support Chrome, Firefox, Edge, and Safari browsers.

Priority:

Medium

Justification:

Ensures compatibility across major platforms.

Mobile OS Compatibility

Requirement:

The mobile app shall support Android 8.0+ and iOS 12+.

Priority:

Medium

Justification:

Covers most active smartphone OS versions.

2.1.3. Operational Scenarios:

Scenario 1: Single Trip Ride (Normal Ride)

This scenario represents a standard one-on-one ride between a passenger and a driver.

Steps:

1. The passenger enters pickup and destination locations.
2. The system calculates fare and finds available drivers.
3. The passenger sends a ride request.
4. A nearby driver accepts the request.
5. The system assigns the driver and shares trip details.
6. The driver arrives, picks up the passenger, and starts the trip.
7. The system provides real-time GPS tracking and ETA updates.
8. Upon reaching the destination, the fare is finalized.
9. The passenger pays through wallet/online/cash.
10. Both passenger and driver submit ratings.

Scenario 2: Carpool Ride Scenario

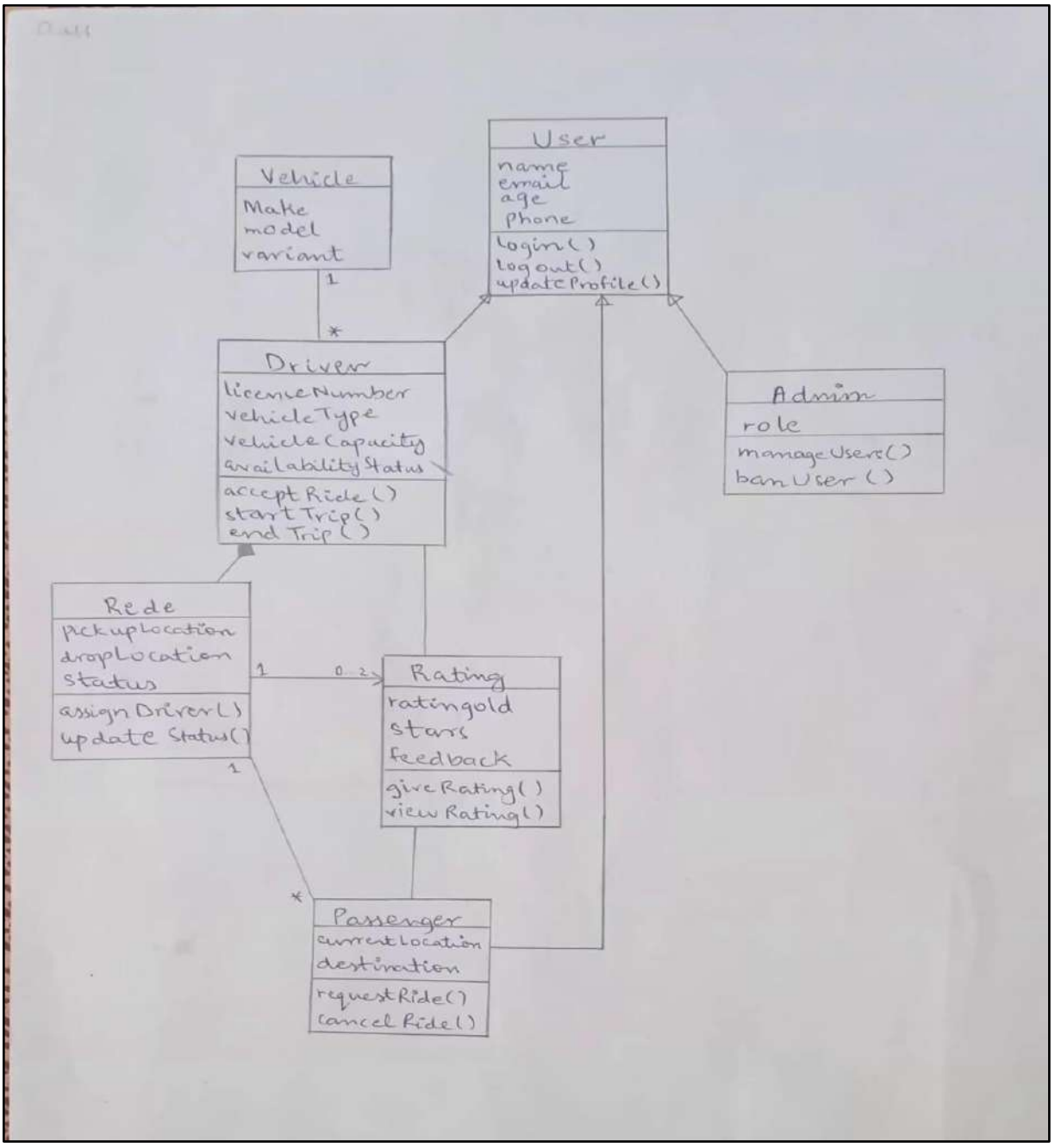
This scenario involves multiple passengers sharing the same vehicle along a similar route.

Steps:

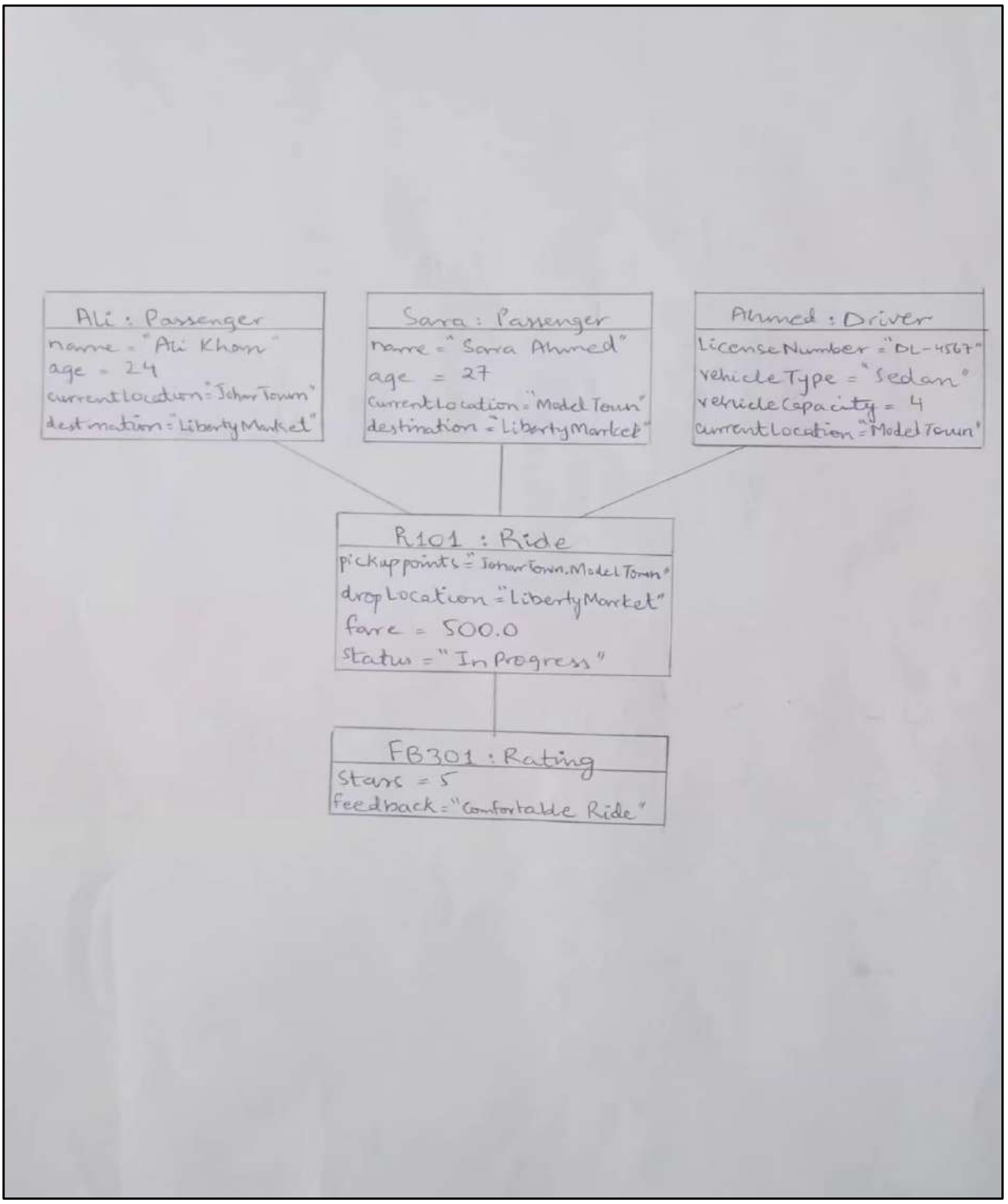
1. The driver posts a carpool ride with available seats and route.
2. Passengers search for matching carpool options.
3. Each passenger requests a seat; the driver accepts based on availability.
4. The system updates remaining seats in real time.
5. Pickup sequence is optimized for all passengers.
6. The driver picks up all passengers as per the route plan.
7. The system adjusts ETAs dynamically for each passenger.
8. Each passenger pays their individual fare share.
9. All passengers leave ratings at the end.

2.2. Structural Model

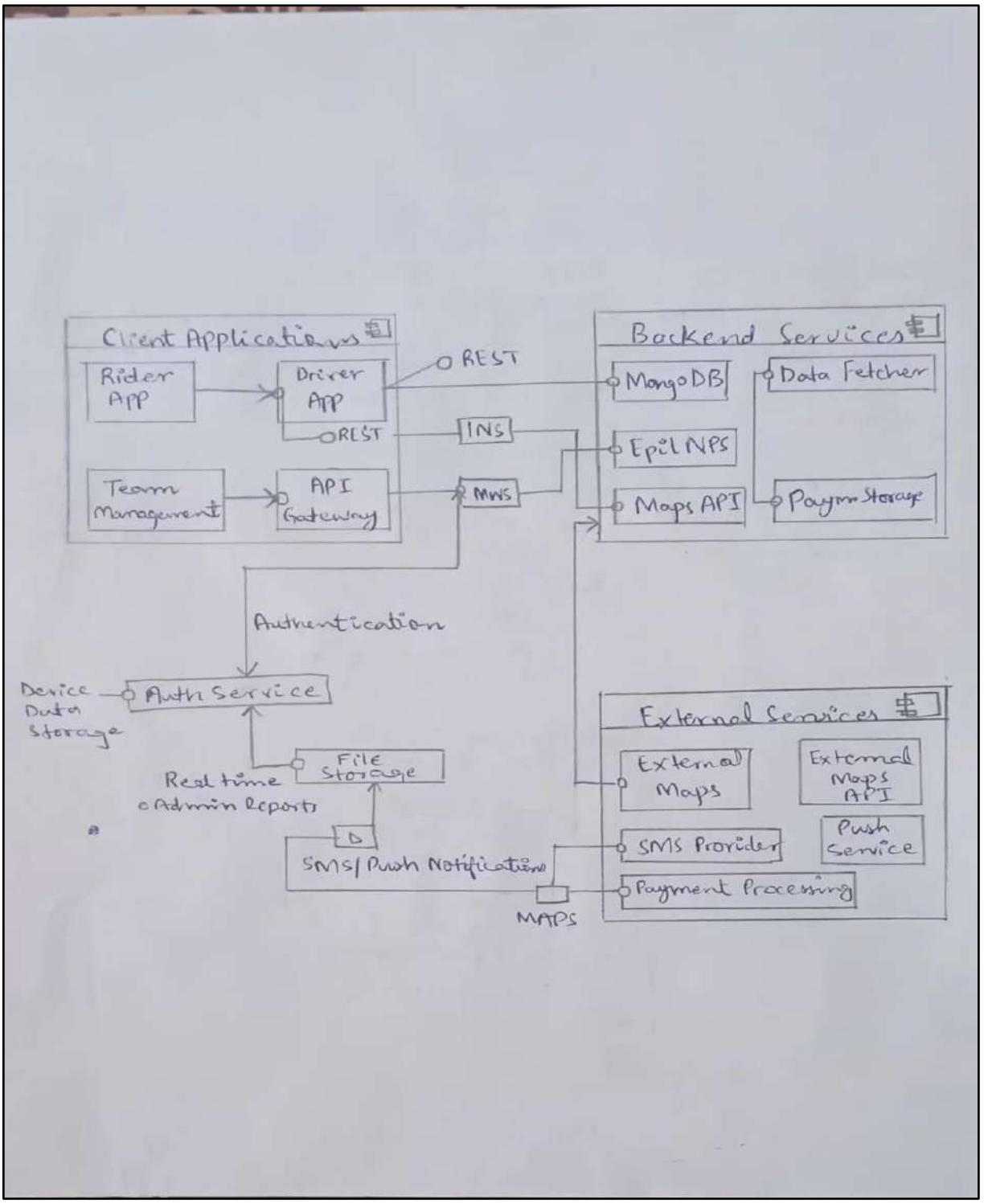
2.2.1. Class Diagram



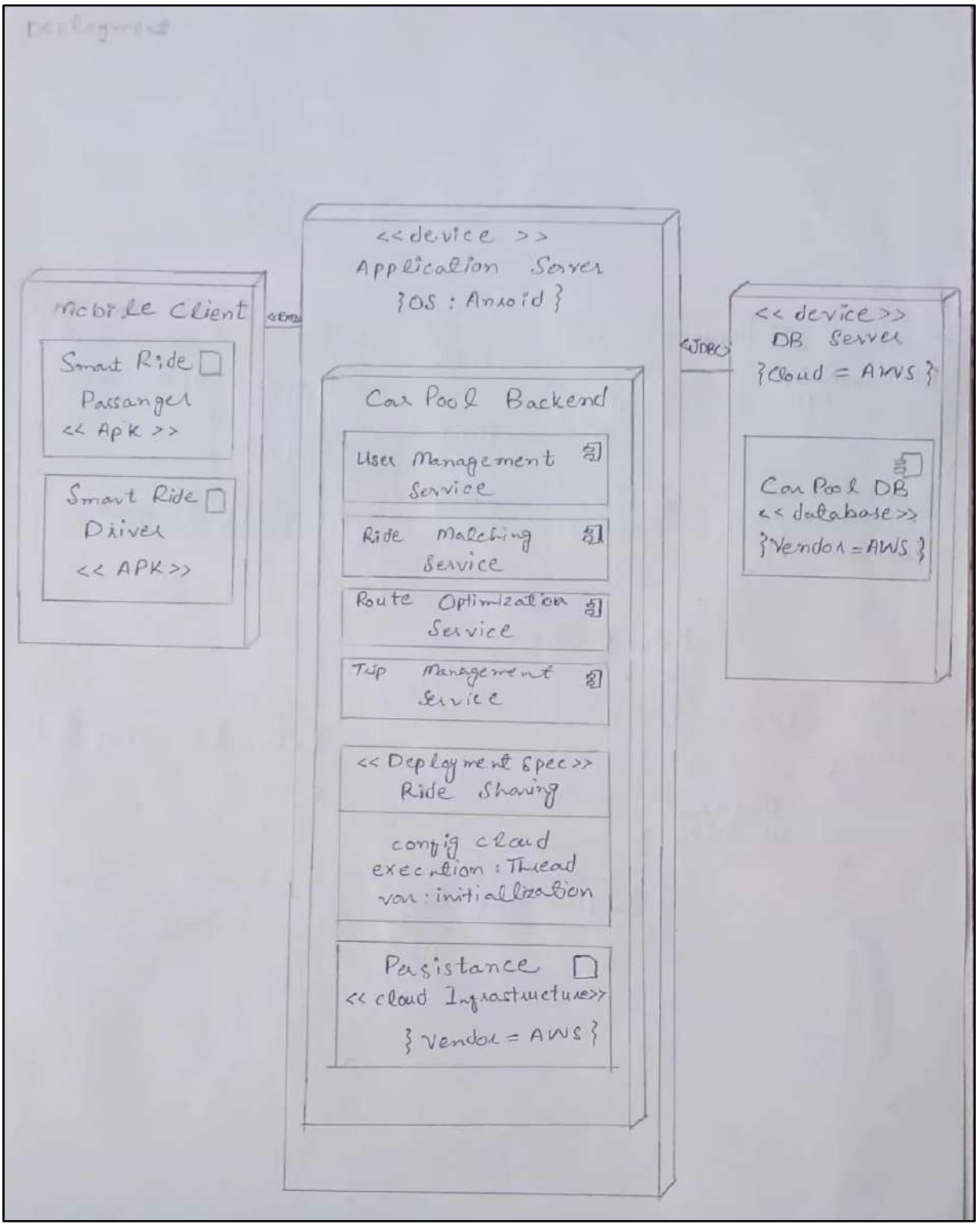
2.2.2. Object Diagram



2.2.3. Component Diagram

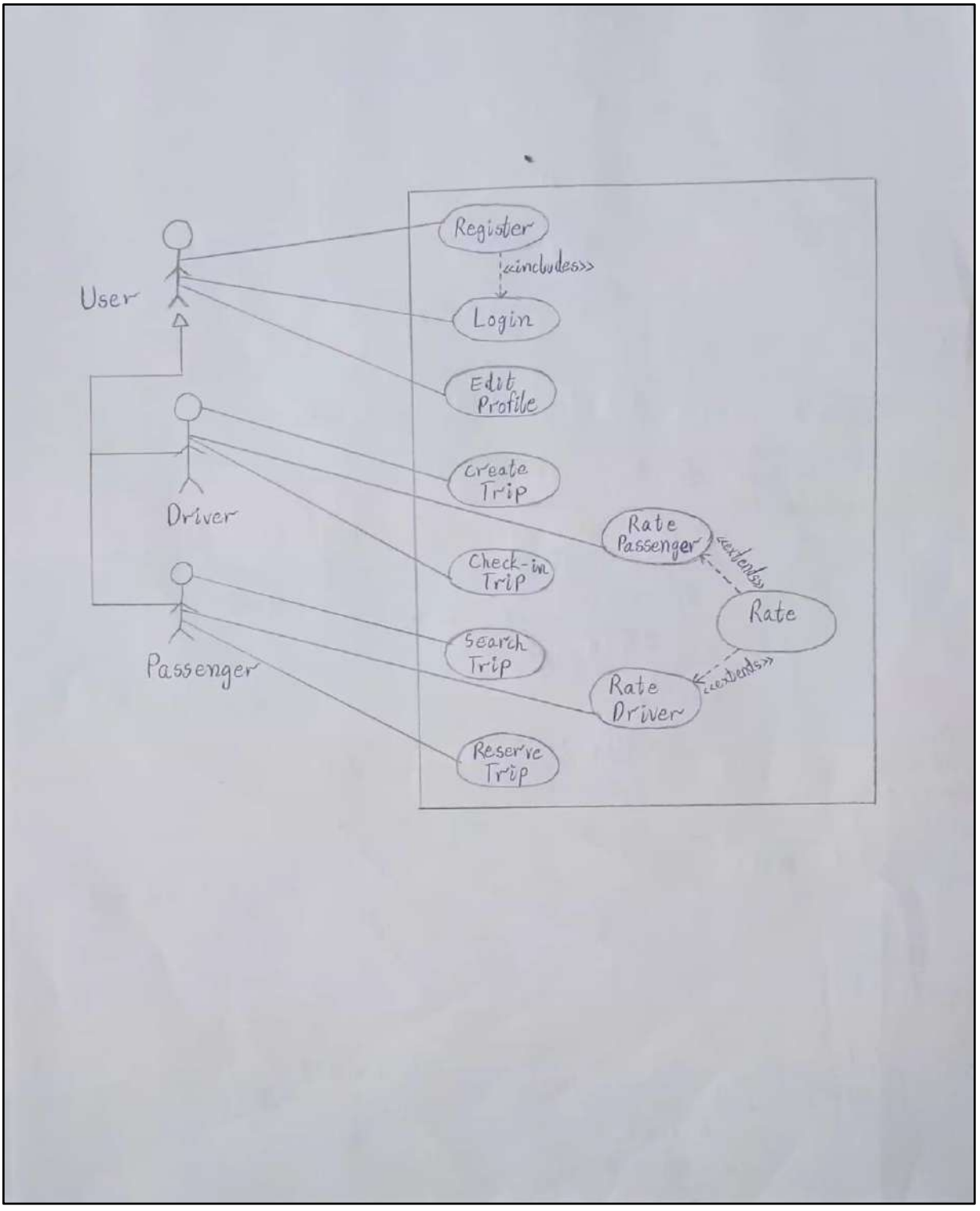


2.2.4. Deployment Diagram



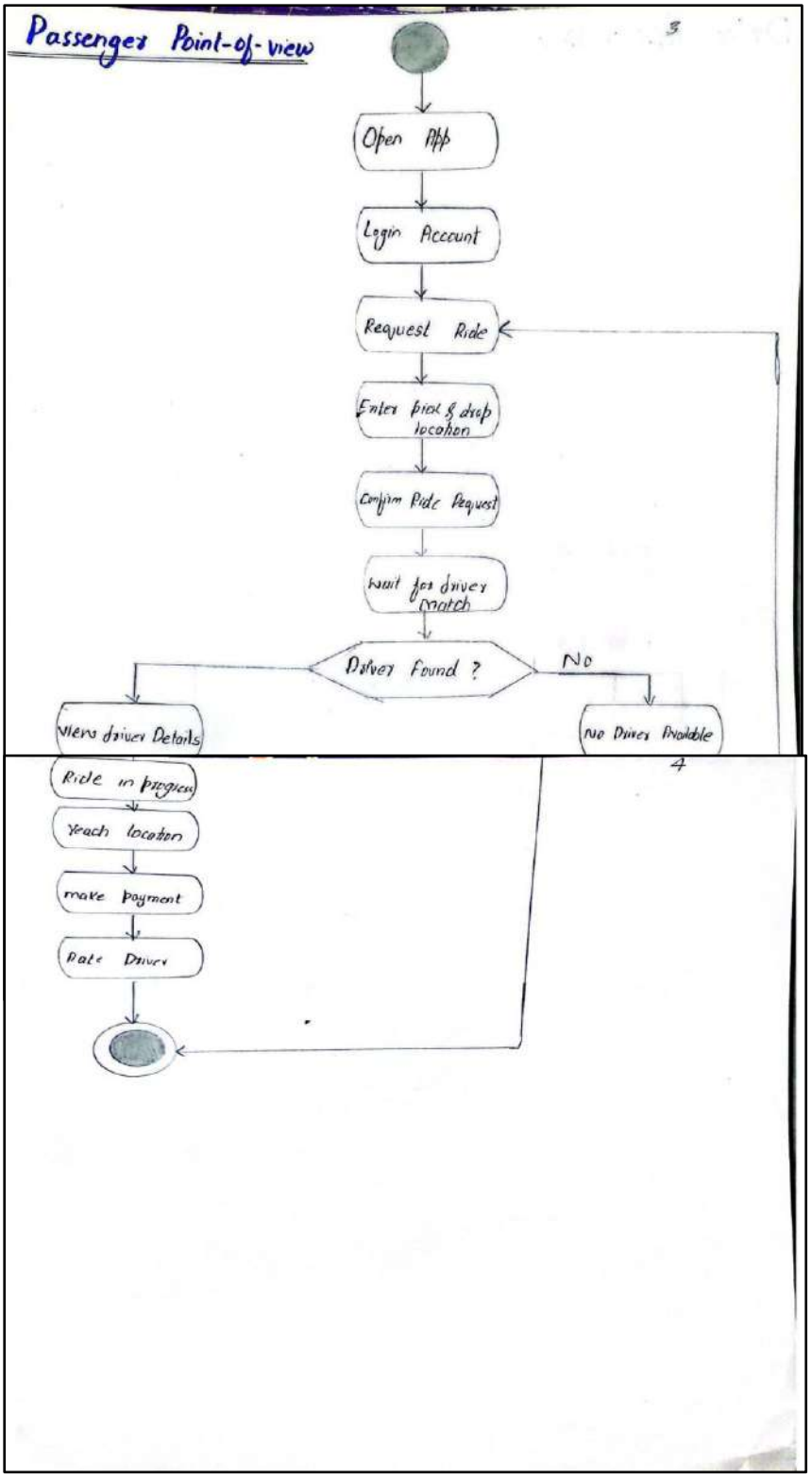
2.3. Behavioral Model

2.3.1. Use Case Diagram

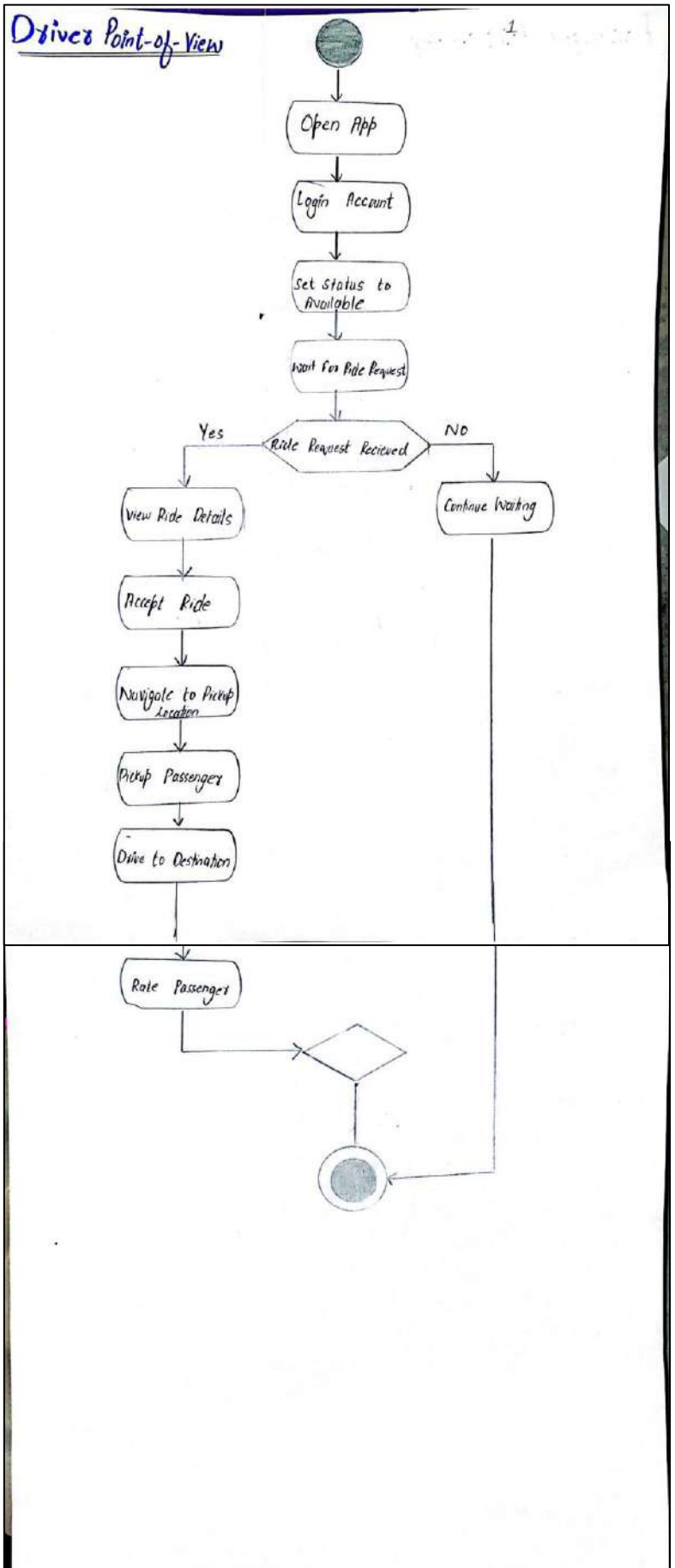


2.3.2. Activity Diagram

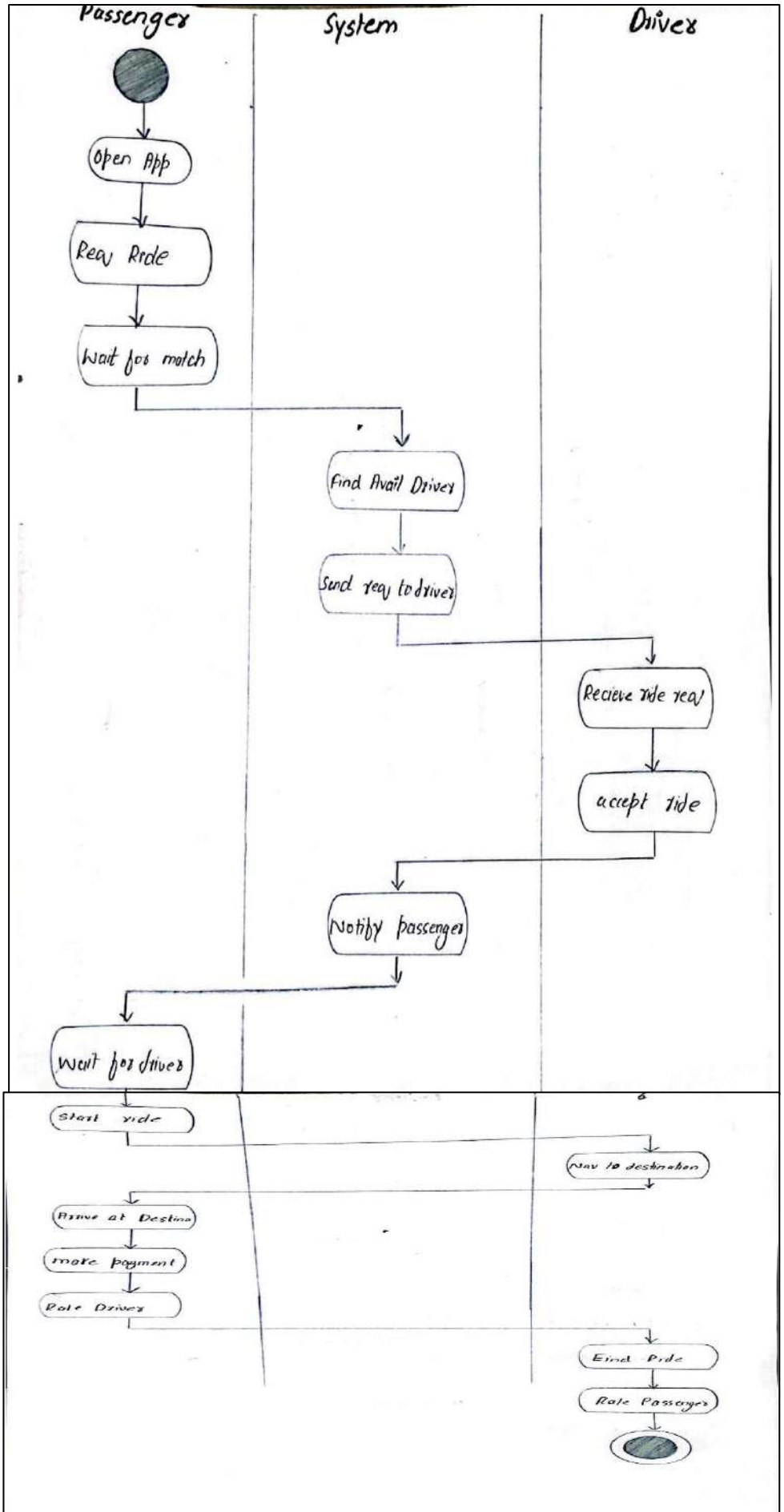
(Passenger's Point of View)



(Driver’s Point of View)

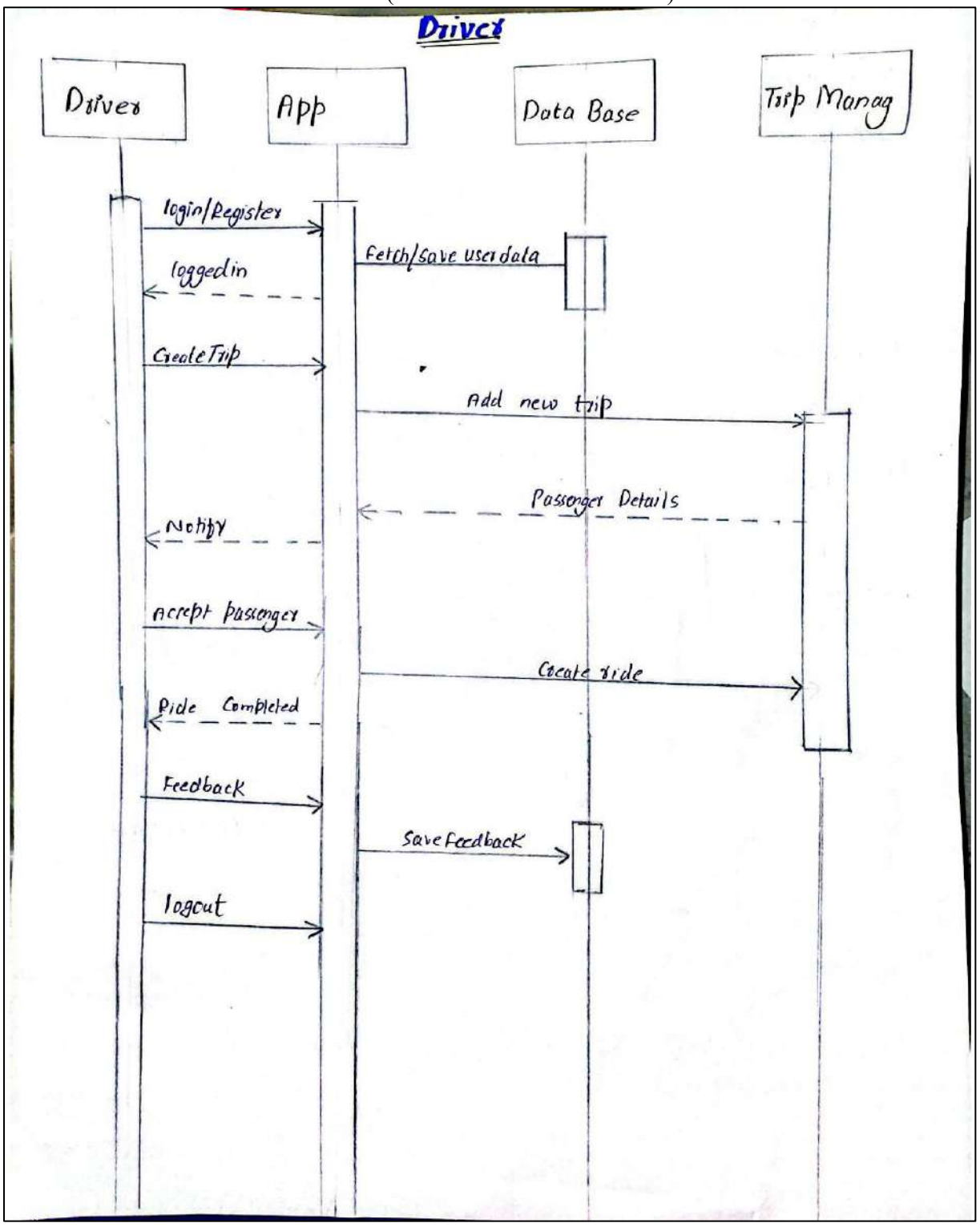


(Combined SwimLane For Trip)

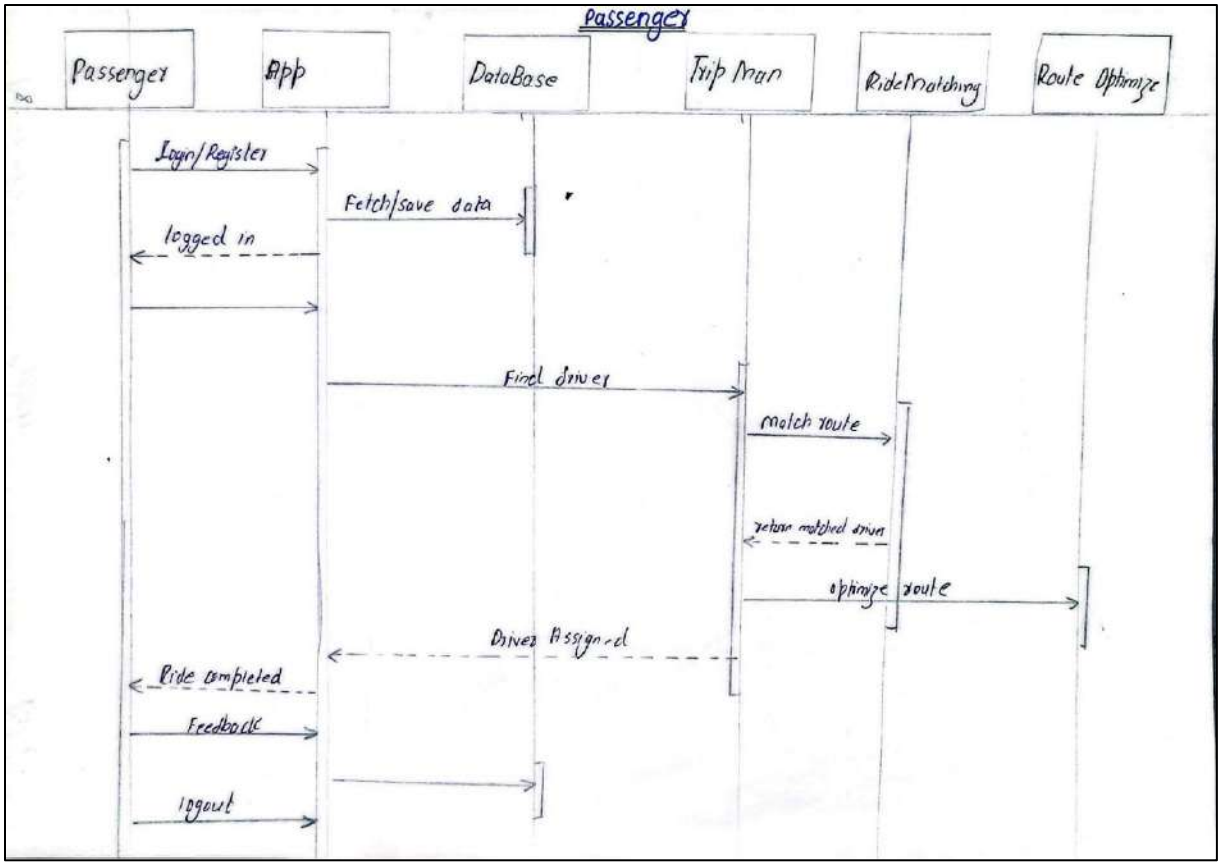


2.3.3. Sequence Diagram

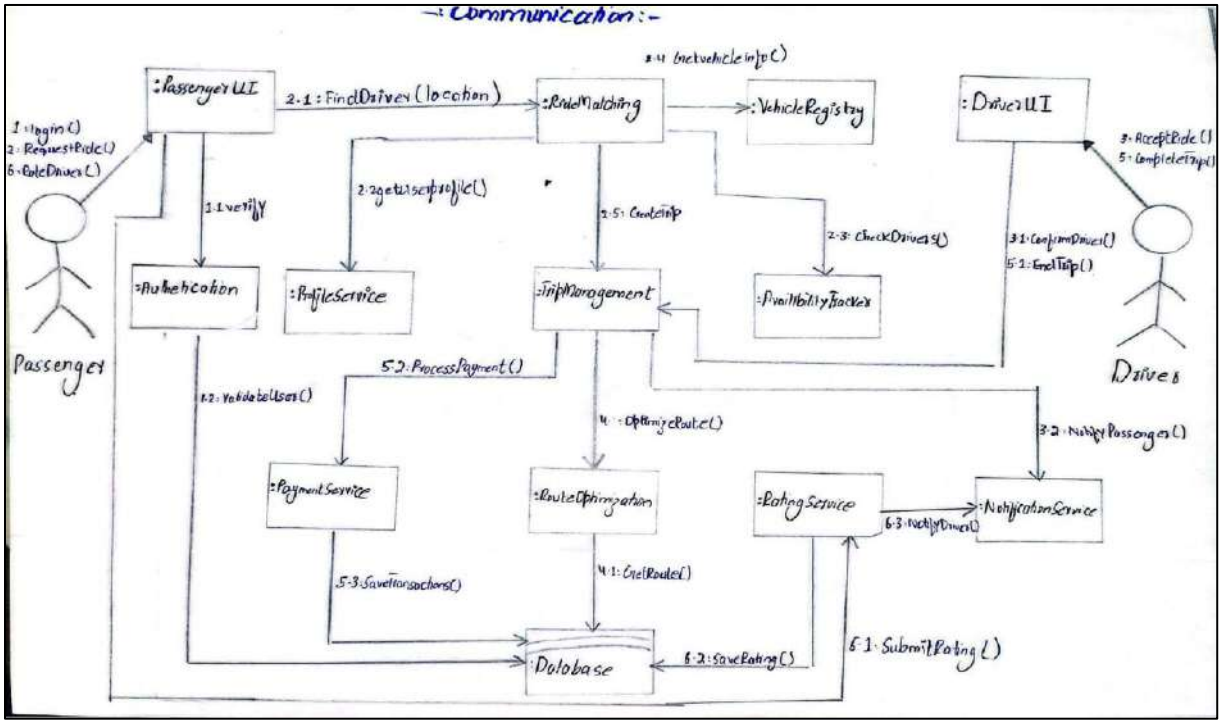
(From Driver's Point of View)



(From Passenger's Point of View)

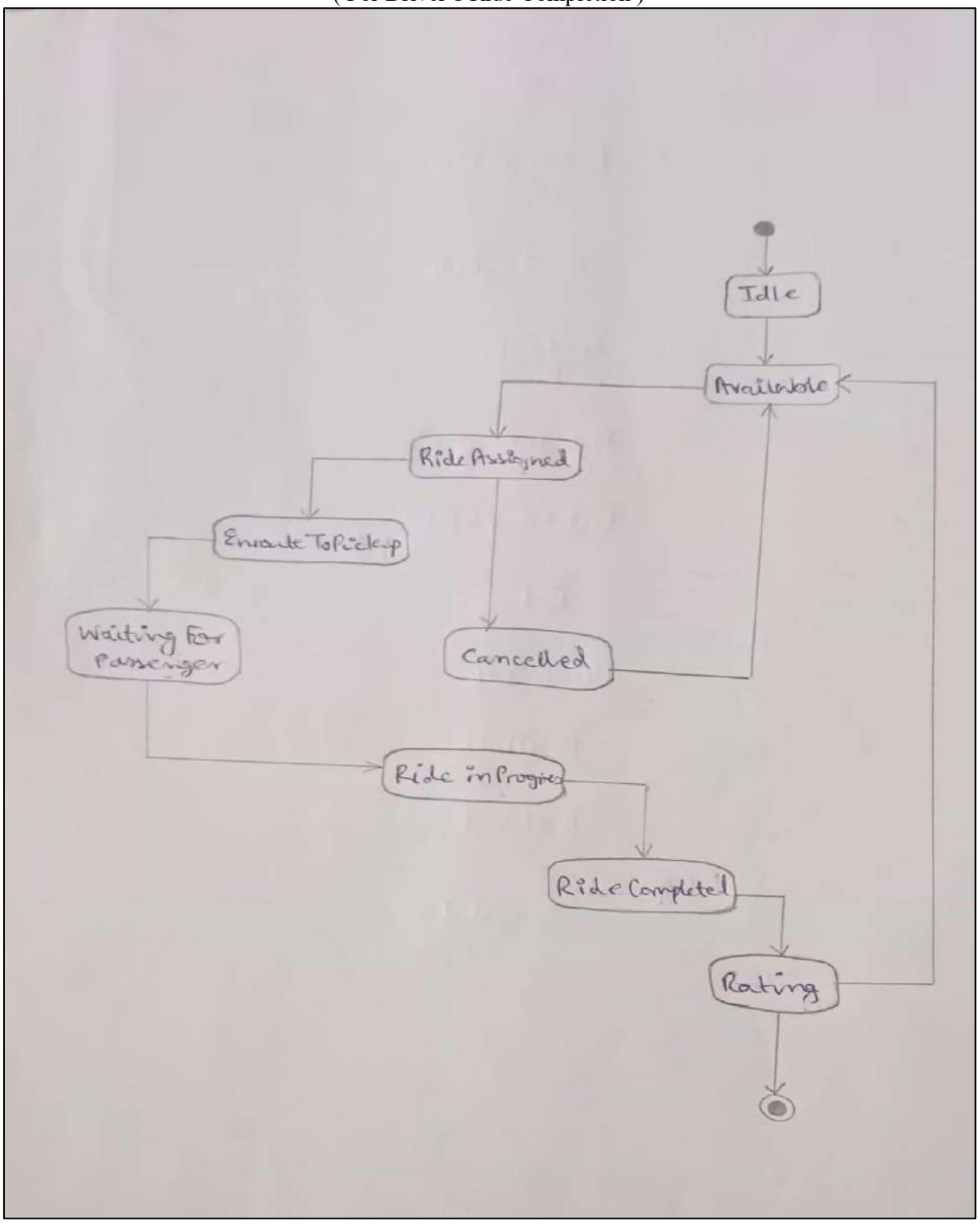


2.3.4. Communication Diagram

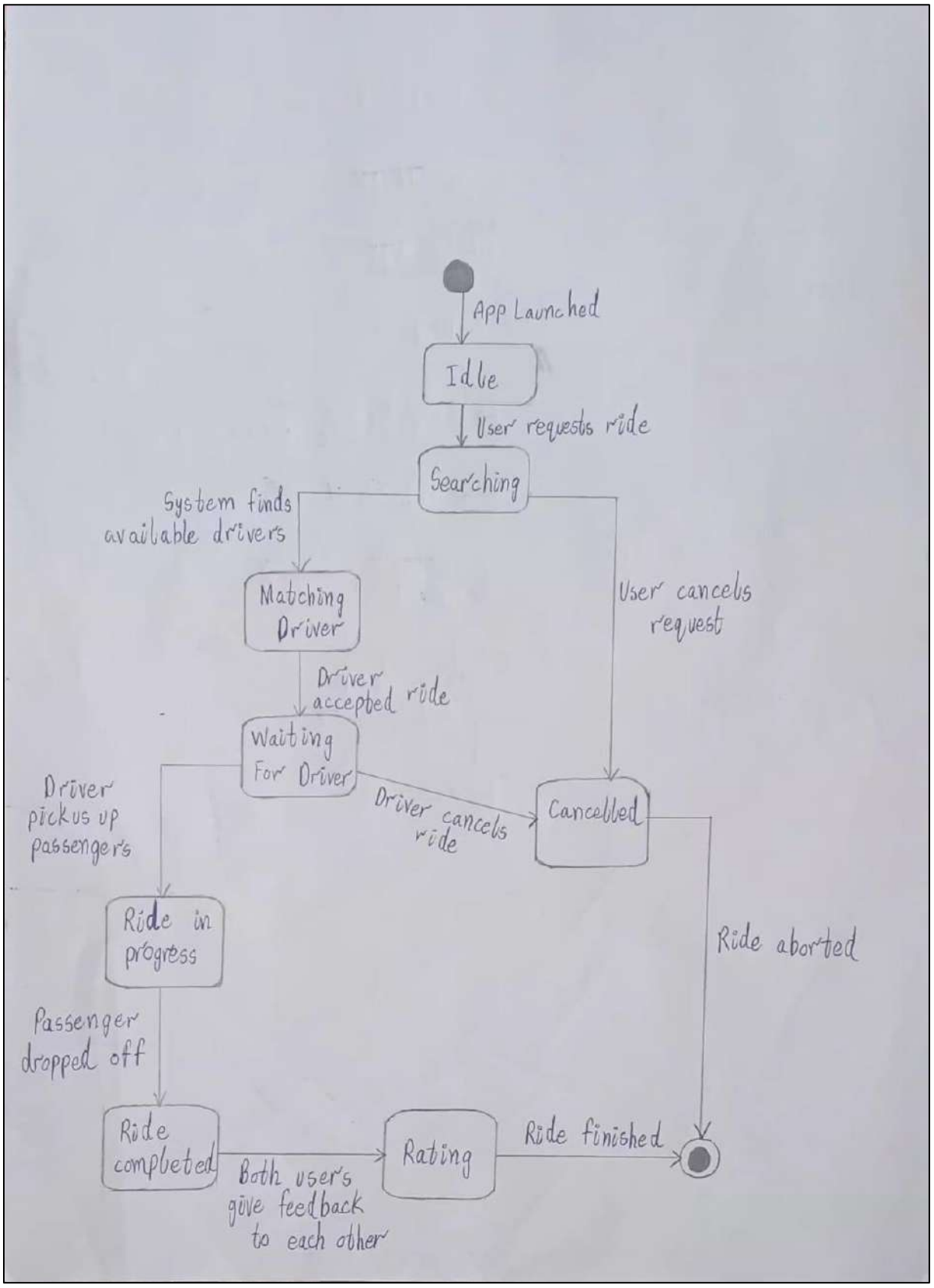


2.3.5. State Diagram

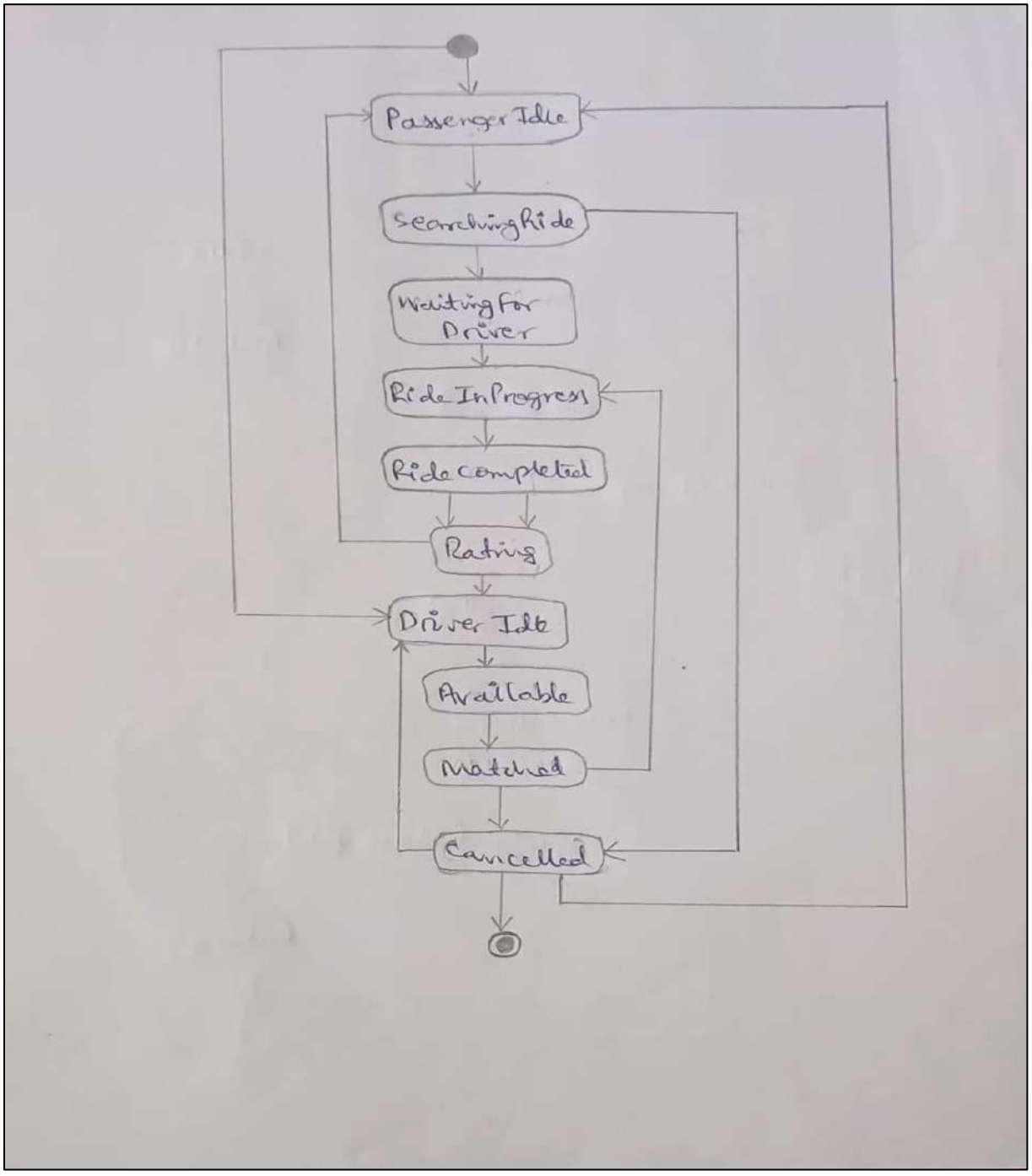
(For Driver's Ride Completion)



(For Passenger's Ride Completion)

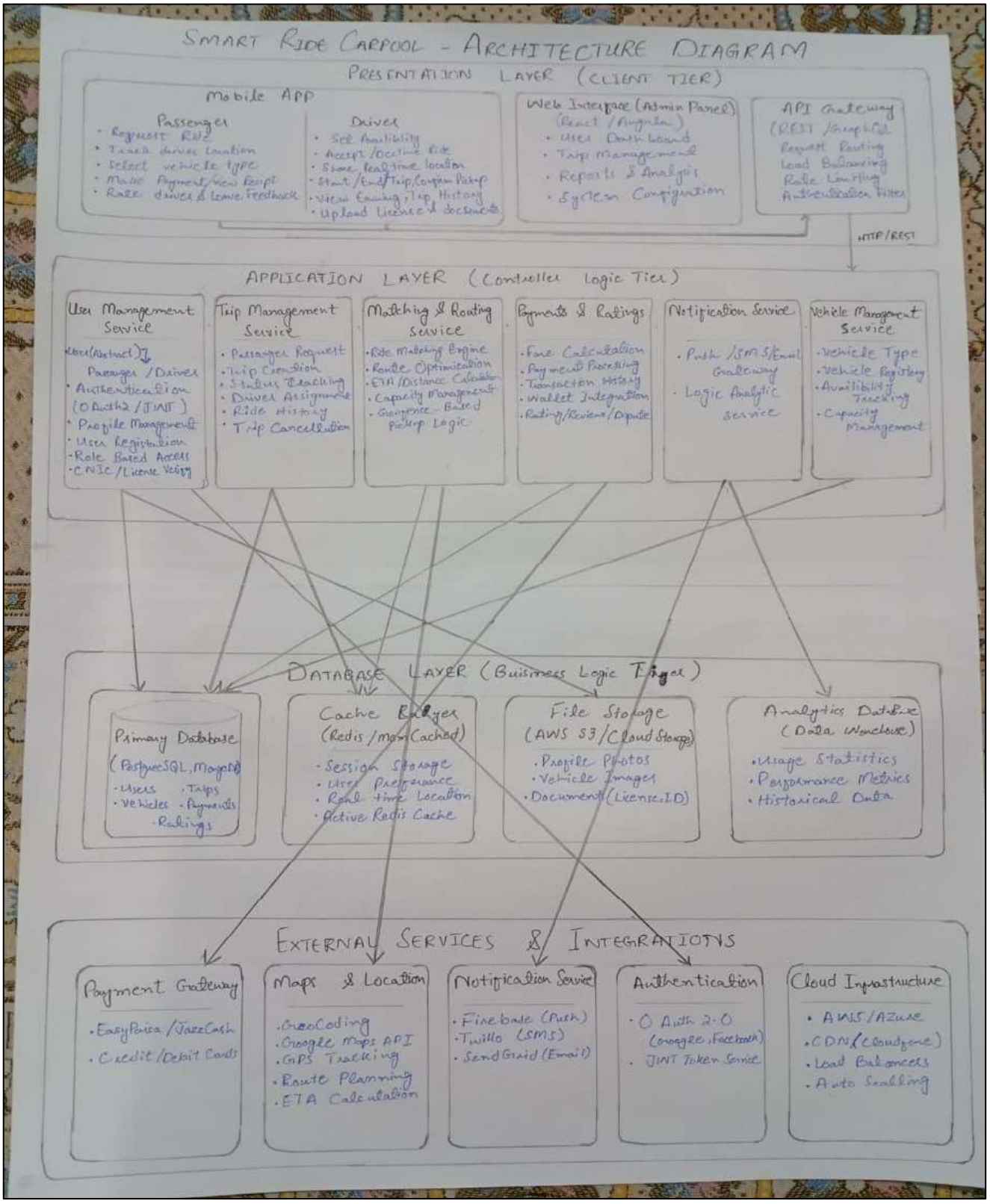


(Combined Point of View)



3. System Design

3.1. System Architecture



1. Client Side:

a) Mobile App (Passenger / Driver Mode)

Unified mobile application supporting dual modes for passengers and drivers on iOS/Android platforms. It connects to API gateway for all backend operations.

Justification:

- Single app reduces maintenance overhead while supporting role-switching.
- **Mobile App → API Gateway** relationship shows it's a client consuming backend services.
- Different responsibilities with same common features justify its dual-mode nature.

b) Web Interface (Admin Panel)

Browser-based administrative dashboard for system management and analytics. It connects to API gateway with admin authentication.

Justification:

- Different interface requirements justify separate components.
- Its relationship shows that it needs backend access for management operations.
- Feature list like User Dashboard, System Configuration justifies need for admin-specific interface

c) API Gateway

Central routing component with REST/GraphQL capabilities, load balancing, rate limiting, and authentication filtering.

Justification:

- It redirects incoming requests from Mobile App and Web Interface to API-routes.
- It acts as a bridge between client and server tiers.
- It provides a single point of entry, security enforcement, and request distribution.

2. Server Side:

a) User Management Service

Handles user lifecycle, authentication, authorization, and profile management with role-based access control.

Justification:

- It shows inheritance structure among users requiring dedicated service.
- Document verification features require file management.
- It shows User Management as the central authentication authority and manages profiles.

b) Trip Management Service

Its core services include managing the entire trip lifecycle, from request creation to trip completion, including status tracking and maintaining trip history. It receives ride requests from the Mobile App via the API gateway and interacts with the Matching & Routing Service to assign drivers and handle other trip-related tasks until completion.

Justification:

- It shows comprehensive trip lifecycle management.
- Trip persistence requires database storage to to ensure data consistency and integrity.
- Real-time status tracking needs caching to ensure fast access updates.

c) Matching & Routing Service

It is responsible for real-time ride matching and route optimization for carpooling, ensuring that drivers are efficiently assigned to passengers in a timely manner. Key features include a “Real-Time Ride Matching Engine” based on the proximity of available drivers, “Multi-Stop Carpool Route Optimization” to calculate optimal routes for drivers with multiple passengers, and “Geofence-Based Pickup Logic” to ensure accurate and efficient pickups.

Justification:

- Time-sensitive operations and complex route optimizations justify the need for

a dedicated service.

- Speed requirements prioritize cache over database to reduce latency.
- Route calculations are offloaded to specialized external-provider providing scalability.
- Advanced location features ensure precise pickup locations and efficient route planning.

d) Payments & Ratings Service

It handles fare calculation, payment processing through a payment gateway, transaction history management, and the mutual rating system (1-5 stars) for passengers and drivers. This service is responsible for processing payments, storing transaction records, and managing user ratings post-trip.

Justification:

- Payments and Ratings are logically grouped together since both are tied to post-trip activities providing one cohesive service.
- Financial transactions need external controllers for security and to ensure compliance with industry standards.
- Transaction records and ratings need persistent storage to maintain financial records and user feedbacks over time.

e) Notification Service

It is a multi-channel messaging system that integrates with push notifications, SMS, and email gateways to deliver messages to users. It also includes logs and analytics capabilities for **admin-side** statistics and insights also.

Justification:

- Multiple channels (push, SMS, email) justify the need for a dedicated service to handle the complexity of interacting with different messaging platforms.
- Message delivery is outsourced to specialized providers (e.g. Firebase for push, Twilio for SMS, SendGrid for email) ensuring scalability and reliability.
- Logs and Analytics capabilities serves a dual purpose: not only tracking message delivery but also providing insights for optimization and performance monitoring.

f) Vehicle Management Service

It handles the management of vehicle types, vehicle registry, availability tracking, and capacity management. It ensures that the system has up-to-date information on vehicle statuses, including availability and capacity for carpool optimization.

Justification:

- Vehicle type and registry-based classification require dedicated management to ensure proper tracking and categorization.
- Availability tracking involves dynamic status updates to reflect real-time availability of vehicles.
- Capacity management is essential for carpool optimization to assign passengers efficiently.

3. Business Logic:

a) Primary Database

It is either relational/document database that stores critical data, including users, trips, vehicles, payments, and ratings. It serves as the central repository for the majority of system data.

Justification:

- It acts as the central data store for the majority of the services, ensuring consistent and reliable access to core data.

b) Cache Layer

It utilizes in-memory caching (via Redis or Memcached) for storing session data, user preferences, real-time location data, and information about active rides. This layer ensures fast access to frequently used data for performance optimization.

Justification:

- Real-time location data requires millisecond access times for efficient updates, making in-memory caching ideal with geocoding.
- Active rides involve temporary data that doesn't require permanent storage.
- Multiple services depend on fast data access, necessitating a shared cache layer

for performance across the system.

- Redis and Memcached are **industry-standard** in-memory caching solutions known for their speed, scalability, and reliability.

c) File Storage

It is used for object storage of profile photos, vehicle images, and documents (e.g., License, ID).

Justification:

- Binary data types (photos, images, documents) are separate from relational data, requiring specialized storage for efficient management.
- AWS S3 offers a scalable and reliable solution for storing large volumes of binary files.
- Document types (such as License and ID) require secure storage due to verification and compliance requirements.

d) Analytics Database

It is a separate database designed to store **usage statistics**, **performance metrics**, and **historical data** for analysis. It is optimized for analytical processing, supporting data analysis and reporting needs without affecting transactional systems.

Justification:

- Data Warehouse" label indicates that this database is intended for analytical processing, not transactional operations, which justifies its separation.
- Separation ensures that intensive queries and data analysis don't impact the performance of transactional DB that handles real-time operations.

4. External Services:

It includes third-party services for **payment gateways**, **maps and location**, **notification delivery**, and **authentication**. These services are integrated into the system to handle specific functions securely and efficiently.

Justification:

- Financial transactions are handled externally through services like EasyPaisa, JazzCash, and credit cards to ensure security and scalability.

- Google Maps or Mapbox APIs are used for external mapping services, providing essential features like geocoding, GPS tracking, route planning, and ETA calculation.

- Multi-channel notification delivery is managed via Firebase (push notifications), Twilio (SMS), and SendGrid (email), ensuring scalable and reliable messaging.

- Authentication is delegated to trusted providers such as OAuth 2.0 (for Google and Facebook login) and JWT Token Services for secure user authentication and token-based session management.

3.2. Design Patterns

Design patterns are proven ways to structure code so it becomes easier to maintain, reuse, scale, and understand.

Our system involves:

➔ Real-time ride request
➔ Multiple drivers receiving request
➔ Driver accepting notification
➔ Passenger receiving updates (driver found, arriving, etc.)

So the most suitable design pattern is:

Observer Pattern

The Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

KEY COMPONENTS:

➔ **Subject** - The object being observed
➔ **Observer** - Objects that watch the subject
➔ **Concrete Subject** - Implementation of subject
➔ **Concrete Observer** - Implementation of observers

BENIFITS:

➔ Loose coupling between components
➔ Dynamic registration of observers
➔ Automatic notification system

➔ Scalable architecture

Core Interfaces

Observer Interface

```
interface Observer {

        void update(String message);

}
```

Subject Interface

```
interface Subject {

    void registerObserver(Observer o);

    void removeObserver(Observer o);

    void notifyObservers(String message);

}
```

Concrete Subject

RideMatchingService

```
class RideMatchingService implements Subject {

    private List<Observer> observers = new ArrayList<>();

    public void registerObserver(Observer o) {

        observers.add(o);

    }

    public void notifyObservers(String message) {

        for (Observer o : observers) {

            o.update(message);

        }

    }
```

```
public void createPassengerrequest(String pickup, String dropoff) {

    String msg = "New Ride Request: " + pickup + " → " + dropoff;

    notifyObservers(msg);

  }

}
```

Concrete Observers

DriverApp

```
class DriverApp implements Observer {

  private String driverName;

  public DriverApp(String name) {

    this.driverName = name;

  }

public void update(String message) {

    System.out.println("[Driver " + driverName + "] Notification: " + message);

  }}
```

PassengerApp

```
class PassengerApp implements Observer {

  private String passengerName;

  public PassengerApp(String name) {

    this.passengerName = name;

  }

public void update(String message) {

    System.out.println("[Passenger " + passengerName + "] Update: " + message);
```

```
  }

}
```

Test Implementation

```
public class TestObserver {

public static void main(String[] args) {

        // Create subject

        RideMatchingService service = new RideMatchingService();

        // Create observers and register them

        Observer d1 = new DriverApp("Akhyar");

        Observer d2 = new DriverApp("Ahmed");

        Observer p1 = new PassengerApp("Rehan");

        service.registerObserver(d1);

        service.registerObserver(d2);

        service.registerObserver(p1);

    // Trigger notification

    service.createPassengerrequest("Gulshan", "Clifton");

  }

}
```

Program Output

```
[Driver Akhyar] Notification: New Ride Request: Gulshan → Clifton

[Driver Ahmed] Notification: New Ride Request: Gulshan → Clifton

[Passenger Rehan] Update: New Ride Request: Gulshan → Clifton
```

4. Test Cases

4.1. Unit Testing

TC-UT-01 – User Registration with Valid Input

Precondition	App must be running; email must be unused.
Assumption	Supported device and stable internet connection.
Test Steps	1. Open the registration page. 2. Enter valid registration details. 3. Click Submit.
Expected Results	Account should be created successfully.
Actual Results	Account created successfully with ID: USR-8847.
Status	Pass
Comment	No issues.

TC-UT-02 – Password Too Short

Precondition	User on registration form.
Assumption	System enforces a min-password length policy.
Test Steps	1. Enter a password shorter than 8 characters. 2. Attempt to submit.
Expected Results	Validation error should appear.

Actual Results	Validation error triggered at 7 characters.
Status	Pass
Comment	Minimum length enforced.

TC-UT-03 – Fare Calculation Formula

Precondition	Mock distance input field available. Fare calculation module is ready.
Assumption	No other system errors interfere with calculation.
Test Steps	1. Input distance = 10 km. 2. Trigger calculate() function.
Expected Results	Correct fare should be computed.
Actual Results	Fare calculated: Rs.150 (Rs15 per km).
Status	Pass
Comment	Formula verified.

TC-UT-04 – Driver Document Upload with Invalid Format

Precondition	Driver must be logged in.
Assumption	Only PNG/JPG formats supported.
Test Steps	1. Attempt to upload a PDF instead of an image.

Expected Results	Error message should be displayed: Invalid file type.
Actual Results	PDF accepted incorrectly.
Status	Fail
Comment	BUG: System incorrectly accepts PDFs.

Test Case: TC-UT-05 – Wallet Balance Deduction

Precondition	Wallet has sufficient balance.
Assumption	Payment module is functional and correctly updates wallet.
Test Steps	1. Call deduct(200).
Expected Results	Balance should decrease accordingly.
Actual Results	Balance reduced from PKR 500 to PKR. 300.
Status	Pass
Comment	Transaction successful.

4.2. Blackbox Testing

TC-BB-01 – Input Buffer Overflow on Location Field

Precondition	Ride request page loaded.
Assumption	Input field has defined maximum character limit.

Test Steps	1. Enter 5000+ characters.
Expected Results	Validation error at max character limit.
Actual Results	Error triggered at 255 characters.
Status	Fail
Comment	BUG: Input validation enforcement needed.

TC-BB-02 – Repeat Payment Click

Precondition	Payment page open.
Assumption	Duplicate prevention enabled.
Test Steps	1. Click Pay rapidly 10 times.
Expected Results	Only 1 transaction should process.
Actual Results	2 transactions ids' were created and processed.
Status	Fail
Comment	BUG: Duplicate prevention not working.

TC-BB-03 – Zero-Length Ride Request

Precondition	Ride request form is loaded.
Assumption	All input fields are mandatory.

Test Steps	1. Submit the form with empty fields.
Expected Results	Error: All fields required.
Actual Results	Correct error message shown.
Status	Pass
Comment	Input validation enforced as expected.

TC-BB-04 – Access First/Middle/Last Items in Trip History

Precondition	Trip history must exist.
Assumption	Scroll/load operations are functional.
Test Steps	1. Scroll to first, middle, and last items.
Expected Results	Items should load correctly.
Actual Results	All 47 items were loaded correctly.
Status	Pass
Comment	Data is intact.

4.3. Integration Testing

TC-IT-01 – Ride Matching Workflow

Precondition	Passenger + Driver both are online.

Assumption	Matching service is active.
Test Steps	1. Passenger requests a ride. 2. Matching service initiates search for nearest available driver.
Expected Results	Nearest driver should be assigned.
Actual Results	Driver DR-1245 assigned in 2.1s.
Status	Pass
Comment	Workflow correct.

TC-IT-02 – Messaging After Match

Precondition	Passenger & Driver matched.
Assumption	Messaging API active.
Test Steps	1. Open the messaging interface b/w Passenger and Driver. 2. Type a message in input field. 3. Hit Submit Button
Expected Results	Message delivered successfully.
Actual Results	Delivery in 1.2 seconds.
Status	Pass
Comment	No latency issues.

TC-IT-03 – GPS Update to Map

Precondition	Driver is moving and app is tracking location.
Assumption	GPS tracking is enabled and functioning.
Test Steps	1. Ensure the driver app is running and location tracking is active. 2. Driver is being moved. 3. Observe the passenger map or monitoring interface for location updates.
Expected Results	Map should refresh in real-time without any errors
Actual Results	Map refreshed every 5 seconds
Status	Pass
Comment	Real-time GPS tracking active; updates observed every 5 seconds.

4.4. Regression Testing

TC-RT-01 – Registration After UI Update

Precondition	Previous version of the registration module is functional and deployed.
Assumption	Regression suite for registration is available and executable.
Test Steps	1. Re-execute all registration test cases UT-01, UT-02, UT-04

Expected Results	No UI issues or functional breakages occur.
Actual Results	All tests passed.
Status	Pass
Comment	Registration module remains backward compatible; no regression issues observed.

TC-RT-02 – Payment Stability After Patch

Precondition	Recent payment-related bug fixes have been deployed.
Assumption	Payment system is functional and no external service issues exist.
Test Steps	1. Perform 10-consecutive payment transactions through the standard payment workflow.
Expected Results	All transactions should be processed once only.
Actual Results	Zero duplicates found.
Status	Pass
Comment	Patch effective; payment stability confirmed.

4.5. Validation Testing

TC-VT-01 – Role-Based Dashboard

Precondition	Valid Passenger and Driver accounts exist and are active.

Assumption	Correct role mapping is defined.
Test Steps	1. Login using the Passenger account. 2. Observe and verify that the Passenger dashboard is displayed. 3. Logout. 4. Login using the Driver account. 5. Observe and verify that the Driver dashboard is displayed.
Expected Results	Each account type should load its respective dashboard without errors and data leakage.
Actual Results	Correct dashboards loaded.
Status	Pass
Comment	Role separation functioning as expected.

TC-VT-02 – Document Verification

Precondition	Driver has successfully uploaded new documents.
Assumption	Verification workflow is active and functioning.
Test Steps	1. Submit the uploaded documents for verification.
Expected Results	Status should become Pending Review.
Actual Results	Status updated in 2.3s.
Status	Pass
Comment	Verification workflow initiated successfully without errors.

4.6. Performance Testing

TC-PT-01 – Ride Matching ≤ 3 Seconds

Precondition	Load testing tool is set up and ready to simulate match requests.
Assumption	The server is operating within its performance capacity
Test Steps	1. Trigger 100 match requests via tool or API
Expected Results	Avg response ≤ 3s.
Actual Results	Avg: 2.8s.
Status	Pass
Comment	Performance target of ≤ 3 seconds was met successfully.

TC-PT-02 – API Concurrency at 5000 Users

Precondition	Load generator tool (e.g., JMeter, LoadRunner) is active and configured for the test.
Assumption	Server infrastructure is scalable.
Test Steps	1. Simulate 5000 users via tool or API
Expected Results	The system should maintain stable performance, with response times within acceptable limits.
Actual Results	CPU usage: 65%, RAM usage: 72%, with no timeouts or service failures.

Status	Pass
Comment	System performance remained stable under load within acceptable limits.

TC-PT-03 – Page Load Time

Precondition	Network connection is stable, with normal bandwidth (e.g. 10 Mbps).
Assumption	Browser/application cache is cleared prior to testing.
Test Steps	1. Open the application and load the dashboard page. 2. Ensure that no background processes (other apps or browser tabs) are affecting the page load time.
Expected Results	≤ 2s avg-load time.
Actual Results	Page loaded in 1.9s.
Status	Pass
Comment	Page load time is within acceptable performance thresholds.

4.7. Load & Stress Testing

TC-LST-01 – 10,000 Ride Requests/Min

Precondition	Load testing tool is ready to simulate high-volume requests.

Assumption	Stress mode is enabled on tool.
Test Steps	1. Simulate 10k ride requests per minute using the load testing tool. 2. Ensure that requests are being processed at the expected rate of 10k requests/minute.
Expected Results	System should remain stable with no crashes or critical errors.
Actual Results	System slowed slightly but remained stable with no crashes.
Status	Pass
Comment	Demonstrated graceful degradation under high load

TC-LST-02 – 500 Payments/Min

Precondition	Payment gateway is ready and active.
Assumption	Peak load scenario.
Test Steps	1. Simulate 500 payments/min.
Expected Results	Zero failures should occur.
Actual Results	495 success; 5 failed due to network timeout
Status	Fail
Comment	BUG: Payment failures under stress.

TC-LST-03 – Server Overcapacity

Precondition	Push the system beyond load threshold alongside ensure the system is operating beyond its defined limits.
Assumption	System has proper error handling in place.
Test Steps	1. Simulate 10,000 concurrent users via the load testing tool.
Expected Results	The system should return a 503 Service Unavailable error and exit gracefully.
Actual Results	Error 503 returned gracefully with no crashes
Status	Pass
Comment	Correct behavior.

4.8. User Acceptance Testing

TC-UAT-01 – Passenger End-to-End Flow

Precondition	A valid passenger account is available, and the user is logged out (if applicable).
Assumption	All modules are integrated and functioning properly.
Test Steps	1. Register a new passenger account. 2. Login with the registered credentials. 3. Request a ride. 4. Complete the ride. 5. Make the payment. 6. Rate the trip out of 5-stars.
Expected Results	Each step should be completed successfully with no

	errors at any stage.
Actual Results	Process completed without errors. Passenger rated 4.7★.
Status	Pass
Comment	The user successfully completed trip and high rating reflects a high level of satisfaction (4.8 stars).

TC-UAT-02 – User Profile Management

Precondition	A valid user is logged in with an active session.
Assumption	Database is properly connected and operational.
Test Steps	1. Update one or more fields in the user profile (e.g. name, phone, email).
Expected Results	Profile updates should be saved correctly and reflected immediately without any errors.
Actual Results	Name & Phone were updated and verified.
Status	Pass
Comment	Data persistency is verified.

4.9. Alpha & Beta Testing

TC-ALPHA-01 – App Without Internet

Precondition	Internet connection is disabled, or the device is in Airplane mode.
Assumption	App is designed to handle offline scenarios
Test Steps	1. Attempt to log in with an active user account while the device is offline (no internet connection).
Expected Results	Error should appear: "No internet connection detected. Please try again later"
Actual Results	Offline message displayed after 3 seconds.
Status	Pass
Comment	Works correctly.

TC-ALPHA-02 – Cancel Ride at All Stages

Precondition	Driver and Passenger are successfully matched.
Assumption	Cancel ride API is functional and capable of processing cancel requests.
Test Steps	1. Attempt to cancel immediately after the ride match (before the ride starts). 2. Attempt to cancel during the ride initiation process. 3. Attempt to cancel during the ride (in progress). 4. Attempt to cancel after the ride is completed (post-ride).

Expected Results	Cancel should work everywhere without errors.
Actual Results	Cancel action fails if attempted ≤ 2 sec of the match, preventing the passenger/driver from canceling the ride early.
Status	Fail
Comment	BUG: Early cancel fails just after match.

TC-BETA-01 – User Friendliness Feedback

Precondition	10 beta testers are available with each having access to the app in the appropriate test environment.
Assumption	App is operating under standard conditions with the expected UX flow, with no custom configurations or experimental features.
Test Steps	1. Beta testers use the app for a set period of time, performing normal tasks. 2. Collect feedback from the testers regarding their experience with the app, focusing on user-friendliness, usability, and interface issues.
Expected Results	1. Actionable feedback should be collected, including both positive insights and areas for improvement. 2. No critical usability issues should be found.
Actual Results	47 feedback items were collected, of which 2 minor issues were identified regarding Dark mode contrast and visibility.
Status	Fail
Comment	Minor UX issues were identified particularly in dark mode.

Appendix A

Definitions

API (Application Programming Interface)
A set of protocols and tools that allows different software applications to communicate with each other.

Carpool
A ride-sharing arrangement where multiple passengers share a vehicle traveling along similar routes to reduce costs and environmental impact.

ETA (Estimated Time of Arrival)
The predicted time when a driver will reach the pickup location or destination.

Fare
The amount of money charged for a ride, calculated based on distance, duration, and other factors.

Geofence
A virtual geographic boundary that triggers specific actions when a device enters or exits the area.

Matching Engine
An algorithm that pairs passengers with suitable drivers based on location, route, availability, and preferences.

Real-time Tracking
Continuous monitoring and display of a vehicle's current location using GPS technology.

Ride Request
A passenger's request for transportation from a pickup location to a destination.

Ride Offer
A driver's announcement of available seats and route for potential passengers.

Session
A period of user activity within the application, typically from login to logout.

Wallet
A digital payment account within the application for storing funds and processing transactions.

Acronyms

API - Application Programming Interface
AWS - Amazon Web Services
CDN - Content Delivery Network
CNIC - Computerized National Identity Card
CPU - Central Processing Unit
DB - Database
ETA - Estimated Time of Arrival
FR - Functional Requirement
GCP - Google Cloud Platform
GDPR - General Data Protection Regulation
GPS - Global Positioning System
HTTP - HyperText Transfer Protocol
HTTPS - HyperText Transfer Protocol Secure
ID - Identification
iOS - iPhone Operating System
IT - Information Technology
JSON - JavaScript Object Notation
JWT - JSON Web Token
KB - Kilobyte
KM - Kilometer
MB - Megabyte
OTP - One-Time Password
PCI - Payment Card Industry
PKR - Pakistani Rupee
RAM - Random Access Memory
REST - Representational State Transfer
S3 - Simple Storage Service (AWS)
SLA - Service Level Agreement
SMS - Short Message Service
SQL - Structured Query Language
SSL - Secure Sockets Layer
TC - Test Case
TLS - Transport Layer Security
UI - User Interface
UML - Unified Modeling Language
URL - Uniform Resource Locator
USB - Universal Serial Bus
UX - User Experience

Abbreviations

Admin - Administrator
App - Application
Avg - Average
BB - Black Box

Config - Configuration
DB - Database
Doc - Document
e.g. - exempli gratia (for example)
etc. - et cetera (and so on)
i.e. - id est (that is)
IT - Integration Testing
LST - Load and Stress Testing
Max - Maximum
Min - Minimum
Msg - Message
NFR - Non-Functional Requirement
PT - Performance Testing
R/W - Read/Write
Req - Requirement
RT - Regression Testing
Sec - Second
UAT - User Acceptance Testing
UT - Unit Testing
vs - versus
VT - Validation Testing

Appendix B

References

1. **Object-Oriented Analysis and Design**
   Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley Professional.
2. **Software Engineering Principles**
   Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
3. **Redis Cache**
   Redis Ltd. (2024). ***Redis Documentation***.
   Available at: https://redis.io/documentation
4. **AWS Cloud Services**
   Amazon Web Services. (2024). ***AWS Documentation***.
   Available at: https://docs.aws.amazon.com/
5. **Google Maps API**
   Google LLC. (2024). ***Google Maps Platform Documentation***.
   Available at: https://developers.google.com/maps/documentation
6. **Firebase Cloud Messaging**
   Google LLC. (2024). ***Firebase Cloud Messaging Documentation***.
   Available at: https://firebase.google.com/docs/cloud-messaging
7. **Twilio SMS API**
   Twilio Inc. (2024). ***Twilio SMS API Documentation***.

Available at: https://www.twilio.com/docs/sms
8. **SendGrid Email API**
Twilio Inc. (2024). ***SendGrid Email API Documentation***.
Available at: https://docs.sendgrid.com/
9. **JazzCash API Documentation**
Jazz Pakistan. (2024). ***JazzCash Merchant Integration Guide***.
Available at: https://www.jazzcash.com.pk/
10. **EasyPaisa API Documentation**
Telenor Microfinance Bank. (2024). ***EasyPaisa Developer Portal***.
Available at: https://easypaisa.com.pk/
11. **OAuth 2.0 Authorization Framework**
IETF. (2012). ***RFC 6749 - The OAuth 2.0 Authorization Framework***.
Available at: https://tools.ietf.org/html/rfc6749
12. **JUnit Testing Framework**
JUnit Team. (2024). ***JUnit 5 User Guide***.
Available at: https://junit.org/junit5/docs/current/user-guide/
13. **Apache JMeter**
Apache Software Foundation. (2024). ***Apache JMeter User's Manual***.
Available at: https://jmeter.apache.org/usermanual/
14. **Ride-Sharing Market Analysis**
Statista Research Department. (2024). ***Ride-Sharing Market Worldwide***.
Available at: https://www.statista.com/outlook/mmo/shared-mobility/shared-rides/ride-sharing/worldwide
15. **Urban Mobility Trends**
McKinsey & Company. (2023). ***The Future of Mobility***.
Available at: https://www.mckinsey.com/industries/automotive-and-assembly/our-insights